



LAB PRACTICE № 11: SHORTEST-PATH AND SPANNING TREES.

COMP1020: OOP and Data Structures

Week 12

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your classes and methods should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables, methods and classes should have meaningful names, and should be followed Java programming convention.
- Academic honesty is required in all work you submit to be graded. Although you can discuss among yourself at high level how to solve the lab problem, you should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions.
- You should upload your .java file(s) to both the Canvas and CMS Autograder **before the end of the laboratory session** unless the instructor gave a specified deadline.
 - (a) No Canvas submission will get 0 point (of course).
 - (b) **No CMS Autograder submission will get point as well!**
- Submit separate *.java file for each Lab problem with the exact file name specified in each lab question (do not add your student ID or your name). For example: **HelloWorld.java**.
- Late submission of lab practice without an approved extension will incur the following penalties:
 - (a) No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
 - (b) The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
 - (c) The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.

Learning Outcome

Topics: Shortest-path and spanning trees.

Outcome: Acquire skills of the use and implementation of graphs algorithms such as Dijkstra's shortest-path algorithm and Kruskal's algorithms of Minimum Spanning Tree in Java.

CMS Autograder

Direct access to this CMS lab is followed: <https://www.cmsvinuni.online>

Note: Due to the current limitation of the Autograder, you must comment out all the `package` declaration before submission.

Problem 1 - Designing Map

John is a student at VinUniversity. One day, he has an idea of drawing a map between 10 buildings on the campus from 0 to 9. However, John does not want to catch the sun by crossing the school yard so he want to go through roads connecting each buildings. In this task, you will follow these instructions to help John design his map so that he can go to the destination with the minimum time:

1. Input:

- In the first line you are given an integer N - numbers of roads.
- On the next N lines there are three distinct integers a , b , w indicating that there is a road between two destination a , b with time w .
- Next line contains an integer c from where John wants to travel to other locations.
- Next line contain q , the number of queries (finding cost) that he wants to perform.
- Next q lines contain an integer v (destination) for which minimum cost is to be found from c .

2. Create a file named `DesigningMap.java` and a public class `DesigningMap`.
3. In the main function, using a `Adjacency List` to represent the relationship of those roads.
4. Implement `Dijkstra` function to find the shortest path.
5. Using the following prompt, complete the problem to print out the minimum time for John.

Listing 1: Template for Problem 1.

```
import java.util.*;

public class DesigningMap {
    static final int MAX = 10; //There are 10 buildings numbered 0 through 9.
    static final int INF = (int)1e9 + 7; //A large number representing ↵
        infinity, used to initialize the distance array.
    static final int[] dist = new int[MAX]; //Stores the shortest distances ↵
        from the start node c to every other node.

    //Each Node represents a building and the cost to reach it.
    //Implements Comparable so nodes can be sorted by weight in a priority ↵
        queue (used in Dijkstras algorithm).
```

```

static class Node implements Comparable<Node> {
    int id, weight;

    public Node(int id, int weight) {
        //TODO: Code here
    }

    @Override
    public int compareTo(Node other) {
        return this.weight - other.weight;
    }
}

static ArrayList<Node> graph[] = new ArrayList[MAX]; //An adjacency list (↔
graph[i]) to store all neighbors of building i with weights.

public static void Dijkstra(int s) {
    //TODO: Implement Dijkstra's algorithm to compute the shortest ↔
    distance from node 's' to all other nodes using a priority queue.
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int N = sc.nextInt();

    for (int i = 0; i < MAX; i++) {
        dist[i] = INF;
        graph[i] = new ArrayList<Node>();
    }
    //Input your graph
    for (int i = 0; i < N; i++) {
        //TODO: Use Scanner to scan the inputs and input your graph
    }

    int c = sc.nextInt();
    Dijkstra(c);
    int q = sc.nextInt();

    for (int i = 0; i < q; i++) {
        int v = sc.nextInt();
        System.out.println(dist[v] != INF ? dist[v] : "NO PATH");
    }
}
}

```

```
7
0 1 4
0 3 8
1 4 1
1 2 2
4 2 3
2 5 3
3 4 2
0
4
1
4
4
5
5
9
7
NO PATH
```

Figure 1: An example of input and output of Problem 1

Problem 2 - Minimum Spanning Tree - Kruskal's algorithm

Good news! Your city VinLand is about to hold the next Summer Olympics Games in four years. The mayor of VinLand is planning to upgrade the city's infrastructure to draw visitors across the globe to the city during the event. His first step is to upgrade the roads between the nine (9) tourist avenues of the city: Olympics Center (OC), North Hotel (NH), Park (PK), Central Mall (CM), Conference Center (CO), Beach (BC), City Center (CC), Movie Theatre (MT), South Hotel (SH) and Airport (AP). The current road network of the city is as follows:

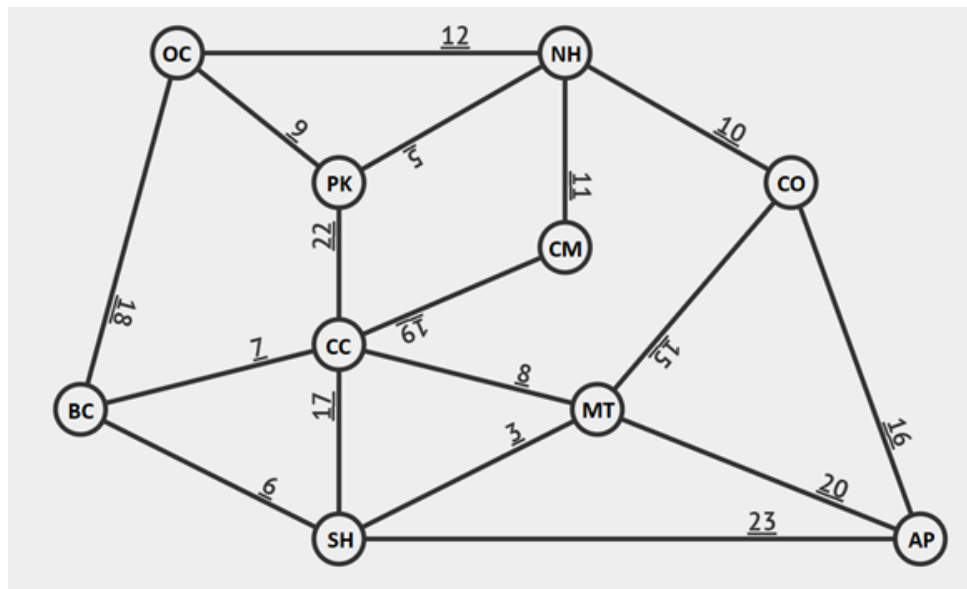


Figure 2: The current road network

However, due to financial constraints as a result of priority being put into constructing stadiums and sports complex, he needs to develop a route of eight roads through the nine avenues and upgrade those roads with the minimal cost. To find this route, implement Kruskal's algorithm with a **union-find** data structure in a `CityPlanner.java` file:

1. Implement a `find()` method that returns the parent of a subset (group). Each tourist avenue starts out in its own **subset** (group), with itself as the **parent**. When we examine a road (edge), we use the `find()` method to check which subset each endpoint (city) belongs to.
2. Implement a `union()` method joins the two subsets: If the two cities belong to **different subsets**, the road can safely be added without creating a cycle. If the two cities are already in the same subset (i.e., have the same parent), adding the road would create a loop, so we skip it.
3. Implement a `findMST()` method:
 - (a) Sort all the edges in ascending order of their weight
 - (b) Pick the smallest edge
 - (c) If the edge results in a cycle, don't include it and increment the index of the resultant set.

Use the following prompt to solve problem 2:

Hint: use built-in function `equals()` to compare String objects in Java

```
The plan to upgrade is as follows:
- Movie Theatre <--> South Hotel      | Cost: 3
- North Hotel   <--> Park              | Cost: 5
- Beach         <--> South Hotel      | Cost: 6
- City Center   <--> Beach            | Cost: 7
- Olympics Center <--> Park           | Cost: 9
- North Hotel   <--> Conference Center | Cost: 10
- North Hotel   <--> Central Mall      | Cost: 11
- Movie Theatre <--> Conference Center | Cost: 15
- Airport       <--> Conference Center | Cost: 16
The MST has the Minimum Cost of 82
```

Figure 3: An example of input and output for problem 2

Listing 2: Template for Problem 2.

```
import java.util.ArrayList;
import java.util.List;
import java.util.Collections;

class Edge implements Comparable<Edge> {
    String v1;
    String v2;
    int weight;

    public Edge(String v1, String v2, int weight) {
        this.v1 = v1;
        this.v2 = v2;
        this.weight = weight;
    }

    // This function orders the Edge objects by weights
    @Override
    public int compareTo(Edge other) {
        return Integer.compare(this.weight, other.weight);
    }
}

class Subset {
    String parent;
    int rank;
}

class Graph {
    List<Edge> edges;
    List<String> nodes;
    int numOfVertices;
```

```

public Graph() {
    this.edges = new ArrayList<>();
    this.nodes = new ArrayList<>();
}

public void addEdge(String v1, String v2, int weight) {
    Edge e = new Edge(v1, v2, weight);
    edges.add(e);
    if (!nodes.contains(v1)) {
        nodes.add(v1);
        numOfVertices++;
    }
    if (!nodes.contains(v2)) {
        nodes.add(v2);
        numOfVertices++;
    }
}

//This function returns the parent of the subset that contains the vertex
public String find(Subset[] subsets, String vertex) {
    int i = nodes.indexOf(vertex);
    //TODO: Your code here
}

//Function to join two subsets that contains x and y
public void union(Subset subsets[], String x, String y) {
    String xroot = find(subsets, x);
    String yroot = find(subsets, y);
    int xroot_index = nodes.indexOf(xroot);
    int yroot_index = nodes.indexOf(yroot);

    //TODO: If one tree ranks lower than the other, attach it under the ←
    other
    //Your code here
    //TODO: If not, choose one and make it the root and increase the rank
    //Your code here
}

public void findMST() {

    //Store the resultant edges
    Edge[] result = new Edge[numOfVertices];
    int e = 0; // Number of edges in result[]
    int i = 0; //Index of sorted edges

    //Sort the edges in ascending order
    Collections.sort(this.edges);

    //TODO: Create V subsets with one element

```

```

// Your code here

//Loop until the number of edges is equal to the number of vertices ←
    minus 1 (E = V-1)
i=0;
while (e < numOfVertices - 1) {
    // TODO: Gets the smallest edge and increase i gradually

    // TODO: If the edge doesn't result in a cycle, add it to the ←
    resultant set and join the two subsets

}
}

System.out.println("The plan to upgrade is as follows:");
int minCost = 0;
for (i = 0; i < e; ++i) {
    System.out.println(
        String.format("- %-15s <--> %-17s | Cost: %d", result[i].←
            v1, result[i].v2, result[i].weight));
    minCost += result[i].weight;
}
System.out.println("The MST has the Minimum Cost of " + minCost);
}

}

public class CityPlanner {
    public static void main(String[] args) {
        Graph g = new Graph();
        g.addEdge("Olympics Center", "North Hotel", 12);
        g.addEdge("Olympics Center", "Park", 9);
        g.addEdge("Olympics Center", "Beach", 18);
        g.addEdge("North Hotel", "Park", 5);
        g.addEdge("North Hotel", "Central Mall", 11);
        g.addEdge("Park", "City Center", 22);
        g.addEdge("City Center", "Central Mall", 19);
        g.addEdge("City Center", "Movie Theatre", 8);
        g.addEdge("City Center", "Beach", 7);
        g.addEdge("City Center", "South Hotel", 17);
        g.addEdge("Beach", "South Hotel", 6);
        g.addEdge("Movie Theatre", "South Hotel", 3);
        g.addEdge("Movie Theatre", "Airport", 20);
        g.addEdge("North Hotel", "Conference Center", 10);
        g.addEdge("Movie Theatre", "Conference Center", 15);
        g.addEdge("Airport", "Conference Center", 16);
        g.findMST();
    }
}

```